**You said:**

give all kafka command

**ChatGPT said:**

# Apache Kafka Commands Cheat Sheet

Here is a practical collection of Kafka commands for learning, Python development, Docker, and production-oriented data engineering. Since you're using **Confluent Platform Kafka 7.9.x with Docker and Kafka UI**, I'll include both standard Apache Kafka commands and Confluent-specific examples.

## 1. Start Kafka using Docker

### Check Docker and Kafka containers

```
docker --version
docker ps
docker ps -a
docker images
docker network ls
```

### Create a Docker network

```
docker network create kafka-net
```

If the network already exists, list it using:

```
docker network inspect kafka-net
```

### Pull Confluent Kafka images

```
docker pull confluentinc/cp-kafka:7.9.0
docker pull confluentinc/cp-zookeeper:7.9.0
```

Use the specific image tag that matches your setup.

### Start Kafka UI

```
docker run -d \
  --name kafka-ui \
  --network kafka-net \
  -p 7000:8080 \
  -e DYNAMIC_CONFIG_ENABLED=true \
  provectuslabs/kafka-ui:latest
```

Open `http://localhost:7000` . Kafka UI still needs a reachable Kafka broker configured before it can display your cluster.

### Manage containers

```
docker start kafka
docker stop kafka
docker restart kafka
docker logs -f kafka
docker exec -it kafka bash
docker rm -f kafka
```

Replace `kafka` with your actual container name.

## 2. Kafka CLI commands: where to run them

Inside a Kafka container, the CLI scripts are commonly located at:

```
/opt/kafka/bin/
```

For Confluent Platform images, use:

```
/usr/bin/
```

For example, enter the container:

```
docker exec -it kafka bash
```

Then locate the tools:

```
which kafka-topics
which kafka-console-producer
which kafka-console-consumer
```

If running commands from your host machine, you need Kafka CLI tools installed locally or you can execute them inside the container.

## 3. Topic management commands

A **topic** is a named stream of records in Kafka.

Assume your broker is accessible at `localhost:9092` .

| Task | Command |
| --- | --- |
| List topics | `kafka-topics --bootstrap-server localhost:9092 --list` |
| Create topic | `kafka-topics --bootstrap-server localhost:9092 --create --topic orders` |
| Describe topic | `kafka-topics --bootstrap-server localhost:9092 --describe --topic orders` |
| Delete topic | `kafka-topics --bootstrap-server localhost:9092 --delete --topic orders` |
| List topic configuration | `kafka-configs --bootstrap-server localhost:9092 --entity-type topics --entity-name orders --describe` |

Create a topic with three partitions and replication factor one (suitable for a single-broker local demo):

```
kafka-topics \
  --bootstrap-server localhost:9092 \
  --create \
  --topic orders \
  --partitions 3 \
  --replication-factor 1
```

Change a topic's partition count:

```
kafka-topics \
  --bootstrap-server localhost:9092 \
```

```
  --alter \
  --topic orders \
  --partitions 5
```

Important: increasing partitions is supported, but decreasing a topic's partition count is not supported through this command.

## 4. Producer commands

A producer sends records to a Kafka topic.

### Start a console producer

```
kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic orders
```

Enter messages, one per line:

```
order-101
order-102
order-103
```

Press `Ctrl+C` to stop.

### Send messages with keys

```
kafka-console-producer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --property parse.key=true \
  --property key.separator=:
```

Example input:

```
order-101:{"amount":500,"status":"created"}
order-102:{"amount":800,"status":"created"}
```

The key helps Kafka choose a partition and preserve ordering for records with the same key under normal partitioning.

## 5. Consumer commands

A consumer reads records from a topic.

### Read existing and new messages

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning
```

### Read only new messages

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders
```

### Read messages with keys

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning \
  --property print.key=true \
  --property key.separator=:
```

### Read a limited number of records

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning \
  --max-messages 10
```

### Display partition and offset

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning \
  --property print.partition=true \
  --property print.offset=true
```

## 6. Consumer group commands

Consumer groups allow multiple consumers to share topic partitions.

| Task | Command |
| --- | --- |
| List groups | `kafka-consumer-groups --bootstrap-server localhost:9092 --list` |
| Describe group | `kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group order-service` |
| Check group state | `kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group order-service --state` |
| Check group members | `kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group order-service --members` |

Consume records as part of a named group:

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --group order-service
```

### Reset consumer offsets

Preview a reset before applying it:

```
kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group order-service \
  --topic orders \
  --reset-offsets \
```

```
  --to-earliest \
  --dry-run
```

Apply the reset only after checking the preview. The group must be inactive for offset resets.

```
kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --group order-service \
  --topic orders \
  --reset-offsets \
  --to-earliest \
  --execute
```

Other reset options include `--to-latest`, `--to-offset 100`, `--shift-by -10`, and `--to-datetime 2026-10-23T10:00:00.000`.

## 7. Partition and offset commands

Partitions divide a topic into independently consumable logs.

### Describe partition details

```
kafka-topics \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic orders
```

### Read records from a specific partition

```
kafka-console-consumer \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --partition 0 \
  --offset 0
```

### Check the end offset

```
kafka-get-offsets \
  --bootstrap-server localhost:9092 \
```

```
  --topic orders \
  --time -1
```

Check the earliest offset:

```
kafka-get-offsets \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --time -2
```

These commands report offsets by partition. An offset is a record's position within its partition, not a global topic-wide position.

## 8. Broker and cluster commands

### Check broker API connectivity

```
kafka-broker-api-versions \
  --bootstrap-server localhost:9092
```

### Check cluster metadata

```
kafka-metadata-quorum \
  --bootstrap-server localhost:9092 \
  describe --status
```

The metadata quorum command applies to KRaft-based clusters. It is not the correct diagnostic for a ZooKeeper-based cluster.

### Check Kafka version

```
kafka-topics --version
```

Or inspect the installed distribution:

```
kafka-run-class kafka.Kafka
```

The latter is not a normal health check and may start a broker process; prefer the version command where supported.

### Check broker logs in Docker

```
docker logs --tail 100 kafka
docker logs -f kafka
```

### Check whether the broker port is listening

On macOS or Linux:

```
nc -vz localhost 9092
```

A successful TCP connection confirms port reachability, not that authentication or Kafka protocol operations will succeed.

## 9. Topic configuration commands

### Set retention to one day

```
kafka-configs \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name orders \
  --alter \
  --add-config retention.ms=86400000
```

### Set the maximum message size

```
kafka-configs \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name orders \
  --alter \
  --add-config max.message.bytes=2097152
```

### Remove an override

```
kafka-configs \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name orders \
  --alter \
  --delete-config retention.ms
```

### List effective configuration

```
kafka-configs \
  --bootstrap-server localhost:9092 \
  --entity-type topics \
  --entity-name orders \
  --describe
```

Remember that retention settings control how long Kafka retains records, not how long a consumer has to process them.

## 10. Kafka performance and troubleshooting commands

### Producer performance test

```
kafka-producer-perf-test \
  --topic orders \
  --num-records 100000 \
  --record-size 100 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:9092
```

### Consumer performance test

```
kafka-consumer-perf-test \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --messages 100000
```

### Check consumer lag

```
kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --describe \
  --group order-service
```

Look at `CURRENT-OFFSET`, `LOG-END-OFFSET`, and `LAG`. High lag can indicate a slow consumer, a traffic spike, insufficient consumer parallelism, or other processing problems.

### Common problems

| Error or symptom | What to check |
|---|---|
| Connection refused | Broker status, listener configuration, port `9092` |
| Unknown topic | Topic name and topic creation |
| Consumer receives no records | Topic, group offsets, producer activity |
| High consumer lag | Processing speed, partition count, consumer health |
| Replication factor error | Available brokers versus requested replication factor |
| Authorization failed | Kafka ACLs and client identity |
| Docker hostname fails | Use the correct internal or external listener address |

## 11. Security and ACL commands

For clusters configured with Kafka ACL authorization, these commands help manage access. They require appropriate administrative permissions.

### List ACLs

```
kafka-acls \
  --bootstrap-server localhost:9092 \
  --list
```

### Grant read access to a consumer principal

```
kafka-acls \
  --bootstrap-server localhost:9092 \
```

```
  --add \
  --allow-principal User:order-app \
  --operation Read \
  --topic orders \
  --group order-service
```

#### Grant write access to a producer principal

```
kafka-acls \
  --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:order-app \
  --operation Write \
  --topic orders
```

#### Remove an ACL

```
kafka-acls \
  --bootstrap-server localhost:9092 \
  --remove \
  --allow-principal User:order-app \
  --operation Write \
  --topic orders
```

For production, configure authentication and TLS properly. These examples assume the broker accepts the relevant administrative connection; a secured cluster may also require a client properties file.

## 12. Kafka Connect commands

Kafka Connect manages integrations that move data between Kafka and external systems.

For a running Connect REST API on port `8083`:

```
# Check Connect status
curl http://localhost:8083/

# List connectors
curl http://localhost:8083/connectors

# Get connector details
```

```
curl http://localhost:8083/connectors/my-connector

# Check connector status
curl http://localhost:8083/connectors/my-connector/status

# Pause a connector
curl -X PUT http://localhost:8083/connectors/my-connector/pause

# Resume a connector
curl -X PUT http://localhost:8083/connectors/my-connector/resume

# Restart a connector
curl -X POST http://localhost:8083/connectors/my-connector/restart

# Delete a connector
curl -X DELETE http://localhost:8083/connectors/my-connector
```

Replace `my-connector` with your connector's actual name. The Connect service must be running and reachable.

### 13. Schema Registry commands

For Confluent Platform with Schema Registry available on port `8081`:

```
# Check service
curl http://localhost:8081/

# List subjects
curl http://localhost:8081/subjects

# List schema versions
curl http://localhost:8081/subjects/orders-value/versions

# Get latest schema
curl http://localhost:8081/subjects/orders-value/versions/latest
```

Schema Registry is useful when implementing Avro, Protobuf, or JSON Schema-based event contracts.

### 14. Kafka commands for Python developers

Since you are learning Kafka with Python, these commands cover the common workflow.

Install the Python client:

```
pip install confluent-kafka
```

Minimal producer:

```
from confluent_kafka import Producer
import json

producer = Producer({
    "bootstrap.servers": "localhost:9092"
})

producer.produce(
    "orders",
    key="order-101",
    value=json.dumps({
        "order_id": 101,
        "amount": 500
    })
)

producer.flush()
```

Minimal consumer:

```
from confluent_kafka import Consumer

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "order-service",
    "auto.offset.reset": "earliest"
})

consumer.subscribe(["orders"])

try:
    while True:
        msg = consumer.poll(1.0)

        if msg is None:
            continue
```

```
        if msg.error():
            print("Error:", msg.error())
            continue

        print(
            msg.key(),
            msg.value().decode("utf-8")
        )
finally:
    consumer.close()
```

The Python client uses `poll()` and returns `None` when no message or event is available during the poll interval. Always check `msg is None` before calling `msg.error()`.

## 15. Most important commands to memorize

These are the core commands you will use repeatedly in development and troubleshooting.

**Create a topic** Copy

```
$ kafka-topics --bootstrap-server localhost:9092 --create --topic orders
```

**List topics** Copy

```
$ kafka-topics --bootstrap-server localhost:9092 --list
```

**Describe a topic** Copy

```
$ kafka-topics --bootstrap-server localhost:9092 --describe --topic orders
```

**Produce records** Copy

```
$ kafka-console-producer --bootstrap-server localhost:9092 --topic orders
```

**Consume records** Copy

```
$ kafka-console-consumer --bootstrap-server localhost:9092 --topic orders --from-beginning
```

**List consumer groups** Copy

```
$ kafka-consumer-groups --bootstrap-server localhost:9092 --list
```

Check consumer lag Copy

```
$ kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group order-service
```

Inspect Docker logs Copy

```
$ docker logs -f kafka
```

**Important for your Docker setup:** `localhost:9092` is appropriate when your host can reach Kafka through its published listener. When running a CLI or Python application inside another container on `kafka-net`, you will generally use the Kafka container's network hostname and its internal listener port instead.

For a complete industry learning path, focus next on partitions and replication, consumer groups and offset management, delivery guarantees, idempotent producers, transactions, Kafka Connect, Schema Registry, security, monitoring, and Kafka integration with Flask.